

Enhance Ahead-Pipelined N-Branch GShare with Tagged Tables

Jun Fan
junfan.me@gmail.com

Abstract—The Next-Generation Championship in Branch Prediction (CBP-NG) measures branch predictors with a Voltage-Frequency-Scaled Speedup (VFS) score that combines prediction throughput, latency, accuracy and energy consumption. Two example predictors in the CBP-NG simulator repository represent the two extremes of trade-off: the TAGE (tage) predictor achieves high accuracy at the cost of high energy consumption, while the ahead-pipelined N-branch GShare predictor (gshareN_ahead) sustains high prediction throughput with low energy consumption.

This paper enhances the ahead-pipelined N-branch GShare predictor by replacing half of its RAM with two ahead-pipelined N-branch tagged tables, achieving accuracy comparable to the TAGE (tage) example predictor while maintaining high prediction throughput and low energy consumption. Training-direction bias, hashing the tagged-table index with the branch path, and expanding the hysteresis to 2 bits further improve accuracy.

On the 168 public CBP-NG training traces, the submitted predictor (gshareN_tagtN_ahead) achieves VFS of 0.9921.

Index Terms—branch prediction, ahead pipelining, tagged predictor, energy efficiency, CBP-NG.

I. INTRODUCTION

The CBP-NG framework evaluates a branch predictor with the Voltage-Frequency-Scaled Speedup (VFS) score [1], which combines three figures of merit (FoM):

- IPC: average prediction throughput in instructions per clock cycle.
- CPI: average number of cycles lost per correct-path instruction because of mispredictions detected at execution.
- EPI: average dynamic energy of the CBP per correct-path instruction.

The predictor with the higher VFS score is considered better. To achieve a higher VFS, one should generally increase IPC, decrease CPI, and decrease EPI.

Based on the example gshareN_ahead predictor in the CBP-NG simulator repository [2], this paper introduces the gshareN_tagtN_ahead (tagtN: tagged table with N-branch prediction style) predictor. The main ideas are:

- **Ahead-pipelined N-branch tagged tables.** Tagged tables [3] are added to override GShare predictions, reducing CPI by improving prediction accuracy. Recent-PC training-direction bias, hashing the path between two consecutive blocks into the tagged-table index, and expanding the hysteresis to 2 bits are introduced (Section IV-C) to further improve prediction accuracy and reduce CPI. The tagged tables are ahead-pipelined; the maximum number of branches predicted per cycle and the

block size are aligned with the base GShare predictor to maintain high prediction throughput (IPC). The number of tagged tables is limited to 2 to keep dynamic energy consumption (EPI) low.

- **RAM floorplan.** With an optimized floorplan (Section IV-D), prediction latency is 1 cycle (300 ps), leading to high prediction throughput (IPC).
- **Energy optimizations.** In addition to gating unnecessary table RAM reads and writes during prediction (Section IV-A) and training (Section IV-B), same-index RAM read gating, tagged-table index placement optimization, and removing heavy buffering on non-critical paths are introduced (Section IV-E) to reduce dynamic energy consumption (EPI).

II. RELATED WORK

Ahead pipelining. Seznec et al. [4] proposed to use information associated with an instruction block to predict the two-block-ahead instruction block address. Seznec & Fraboulet [5] proposed ahead-pipelining the instruction block address generation (IAG), using in-flight information from intermediate instruction fetch blocks that was not available at initiation, in order to maintain accuracy. Cai et al. [6] explicitly tag each counter in ahead-pipelined TAGE with its corresponding missing history (a secondary tag), instead of considering all paths among N branches between when the prediction starts and when the prediction is needed, in order to reduce energy consumption. The submitted predictor utilizes ahead-pipelining for high prediction throughput. The GShare component considers all paths, but the tagged tables do not. The tagged tables do not add a secondary tag; instead they use the current block's PC and history for tag matching.

Tagged-table predictors. The PPM-like predictor [3] uses multiple partially-tagged prediction tables indexed with different history lengths. At prediction time all tables are indexed, and the final prediction is determined by the table with the longest history that matched. The TAGE family [7]–[13] uses multiple partially-tagged tables indexed with history lengths forming a geometric series, with an optimized update/allocation policy. The prediction is determined by choosing between the longest matching table and the second-longest matching table (or the longest matching table that is not low-confidence). TAGE can also be enhanced with other components such as a Statistical Corrector and a loop predictor. The submitted predictor uses tagged tables, but for energy efficiency the number of tables is limited and multiple

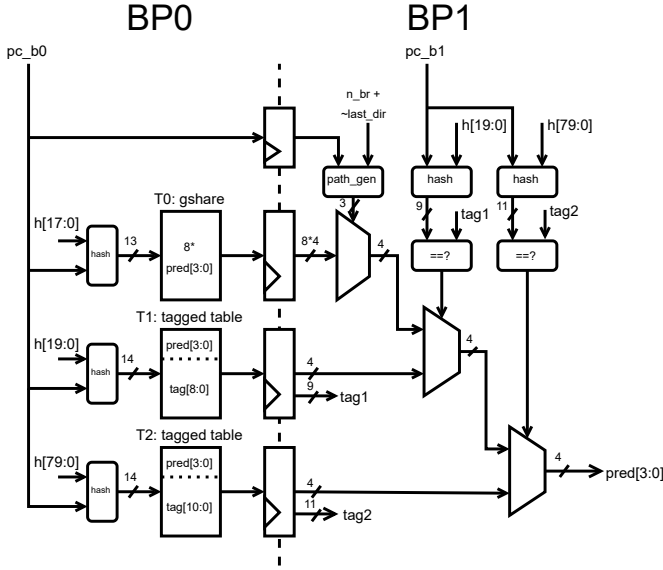


Fig. 1. The submitted gshareN_tagtN_ahed predictor.

branch prediction results are stored in one entry associated with one tag. The final prediction is determined only by the longest matching table.

III. DESIGN OVERVIEW

As shown in Figure 1, the submitted predictor has three components: the GShare (T0) and two tagged tables (T1 and T2). The GShare (T0) is a scaled-down version of gshareN_ahed in the CBP-NG simulator. The original gshareN_ahed predictor produces up to $N = 7$ branch predictions (at most one taken) per cycle with a block size of 1024 instructions. The submitted predictor produces up to $N = 4$ branch predictions (at most one taken) per cycle with a block size of 256 instructions.

The example gshareN_ahed predictor predicts directions for N branches rather than for N instructions, assuming the BTB can interoperate with this N -branch style of prediction. The submitted predictor inherits this behavior.

As shown in Figure 2, each entry in the GShare table provides $\text{PATH} \times N = 8 \times 4$ counters, while each entry in a tagged table provides $N = 4$ counters and one tag. In GShare (T0) and the first tagged table (T1), each counter is 3 bits: one bit is the prediction bit (1'b1 taken, 1'b0 not-taken) and the other 2 bits are the hysteresis bits (2'b11 strong, 2'b00 weak). For timing reasons, each counter in the second tagged table (T2) is 2 bits (not 3 bits): one bit is the prediction bit and the other is the hysteresis bit (1'b1 strong, 1'b0 weak).

The base GShare (T0) has 8192 entries; the first tagged table (T1) has 16384 entries with a 9-bit tag per entry; the second tagged table (T2) has 16384 entries with an 11-bit tag per entry. The total size of the prediction, hysteresis, and tag bits is 176 KB.

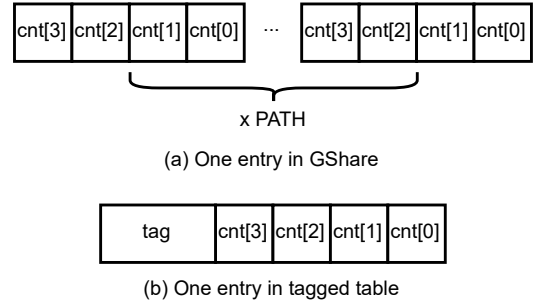


Fig. 2. One entry in GShare and tagged tables.

IV. DETAILED PREDICTOR OPERATION

A. Prediction

As shown in Figure 1, the overall prediction happens in two stages, BP0 and BP1. Instead of indexing the prediction RAM with the address of the current block B1 at the BP1 stage, it is indexed with the address of the previous block B0 at the BP0 stage. The prediction information (tag and prediction bits) is read at BP0 and then flopped into BP1. At the BP1 stage, the final prediction is made for the current block B1.

At the BP0 stage, the indices of GShare and the tagged tables are computed by hashing the address of the previous block B0 with different lengths of global history. The history length is 18-bit, 20-bit, and 80-bit for T0, T1, and T2 respectively. (8×4) bits of prediction are read from T0; a 9-bit tag and 4-bit prediction are read from T1; and an 11-bit tag and 4-bit prediction are read from T2. These prediction and tag bits are flopped into BP1.

At the BP1 stage, the (8×4) bits from T0 are muxed by the path out of block B0 to generate 4 predictions for 4 branches. The path is computed by adding the previous block address, the number of conditional branches, and the last branch direction in the previous block, as in the example gshareN_ahed predictor. For $N = 4$ branches there are at most $N + 1 = 5$ paths out of the previous block B0. To keep the number of paths a power of 2¹, (8×4) bits are read instead of (5×4) bits. There is no path selection for the tagged tables; instead, the tag is matched against the hash of the current block B1's address and the current global history.

The final prediction at BP1 is determined by the matched table with the longest history. GShare is considered always matched, so if no matched entry exists in the tagged tables the GShare prediction is used. The matched table provides predictions for all $N = 4$ branches.

B. Training

For each component (T0, T1, and T2) the prediction and tag bits are stored in one RAM while the hysteresis bits are stored in another. The hysteresis RAM is further divided into

¹The prediction-bits RAM is indexed by $\text{hash}(\text{history}, \text{pc})$ and the hysteresis-bits RAM is indexed by $\{\text{hash}(\text{history}, \text{pc}), \text{path}\}$, so the path need to be a power of 2; only hysteresis bits on the actual path are read and updated after prediction to save energy.

TABLE I
COMPARISON OF CUMULATIVE GSHAREN_TAGTN_AHEAD CONFIGURATIONS WITH THE CBP-NG EXAMPLE PREDICTORS ON THE 168 PUBLIC TRAINING TRACES.

Predictor	VFS	IPC	CPI	EPI	MPKI	P1 Cycle	P2 Cycle	RAM Size
tage (CBP-NG example)	0.8728	5.769	0.05489	1265.5	5.489	0.937	1.863	31 KB
gshareN_ahead (CBP-NG example)	0.9736	9.197	0.06237	243.1	6.930	0.943	0.943	128 KB
tage (CBP2025 TAGE-SC w/o SC)	—	—	—	—	4.430	—	—	142.8 KB
gshareN_ahead (half-size of CBP-NG example)	0.9675	8.692	0.06478	135.4	7.198	0.813	0.813	64 KB
+ first tagged table (T1)	0.9833	8.792	0.05506	194.7	6.118	0.903	0.903	92 KB
+ second tagged table (T2)	0.9909	8.875	0.04861	276.4	5.401	0.903	0.903	130 KB
+ training-direction bias	0.9911	8.875	0.04838	278.5	5.376	0.903	0.903	130 KB
+ tagged-table index hashed with path	0.9912	8.876	0.04829	279.2	5.365	0.967	0.967	130 KB
+ 2-bit hysteresis (full gshareN_tagtN_ahead)	0.9921	8.893	0.04705	312.6	5.228	0.96	0.96	176 KB

Note: Rows in the lower block are cumulative. Each row includes all components above it and adds the component in that row. For example the row of "+ training-direction bias" includes base gshareN_ahead, first tagged table (T1), second tagged table (T2) and training-direction bias. The last row is the submitted gshareN_tagtN_ahead with all components.

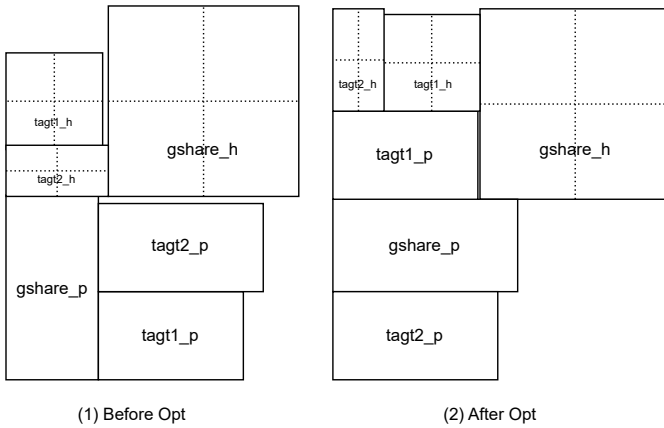


Fig. 3. RAM floorplan of gshareN_tagtN_ahead before and after optimization. *_p are prediction and tag bits RAMs, *_h are hysteresis bits RAMs.

$N = 4$ banks, one per predicted branch, and is read only when the corresponding branch is mispredicted.

On a misprediction, the prediction and tag RAM of the longest matching table is updated only when its hysteresis is weak, with the prediction bit of the mispredicted branch inverted. T1 is allocated when the longest matching table is GShare (T0), and T2 is *always* allocated when the longest matching table is T0 or T1.

The hysteresis RAM of the longest matching table is updated on every prediction. On a misprediction, the hysteresis bits are read and decreased by 1. On a correct prediction, to save energy, the hysteresis bits are not read and are set to $2'b11$ for GShare (T0), to $2'b10$ for the first tagged table (T1), and to $1'b1$ for the second tagged table (T2).

When no misprediction occurs, there is no read on the hysteresis RAM and no write on the prediction and tag RAM, so the write on the hysteresis RAM and the read on the prediction and tag RAM can be overlapped in the same cycle, and no extra cycle is needed. When a misprediction occurs, an extra cycle is needed to read the hysteresis RAM and write

the prediction and tag RAM.

C. Accuracy Optimizations

Recent-PC Training-Direction Bias. Training PCs and their directions are tracked in a two-entry direct-mapped training-direction bias table. Each entry is indexed by the lowest bit of the (skewed) branch PC and tagged by the remaining bits. Each entry holds a 3-bit saturating counter, where counter[2] is the direction and counter[1:0] is the hysteresis. When the longest matching table is GShare (T0) and the prediction is wrong, if the mispredicted PC hits the entry and the counter is strongly biased ($3'b000$ as strongly not-taken or $3'b111$ as strongly taken), the recorded direction is used to train the predictor instead of the actual direction. MPKI is reduced from 5.401 to 5.376 (-0.46%).

Tagged-Table Index with Path. When tagged tables T1 and T2 are read at the BP0 stage, hashing the path (which is used to mux the GShare prediction at BP1) into the prediction RAM index helps de-alias same consecutive blocks reached by different paths. MPKI is reduced from 5.376 to 5.365 (-0.20%).

Hysteresis of 2-bit. Expanding the hysteresis from 1 bit to 2 bits in GShare (T0) and the first tagged table (T1) reduces MPKI from 5.365 to 5.228 (-2.55%).

D. Timing Optimization

RAM Floorplan. As shown in Figure 3, long routing distances between prediction RAMs (gshare_p, tagt1_p, and tagt2_p) incur a prediction latency of 1.17 cycles. Consolidating the prediction RAMs vertically into a more compact layout reduces the prediction latency to 0.96 cycles, thereby sustaining high prediction throughput.

E. Energy Optimizations

GShare RAM read-enable gating. When the GShare prediction RAM index is the same for the current and previous block and the contents of that index have not been updated since the previous read, the GShare prediction can be reused

from the pipeline register instead of reading the RAM again. EPI is reduced from 320.7 to 315.5 (−1.62%).

T1 index located at the T1 prediction RAM. The T1 index generated at the BP0 stage is used to read the T1 prediction RAM. To reduce wire cost, the T1 index register is located near the T1 prediction RAM rather than other RAMs. EPI is reduced from 315.5 to 313.7 (−0.57%).

Remove heavy buffering on non-critical paths. Heavy buffering and large gate sizes are not required to reduce non-critical path latency. EPI is reduced from 313.7 to 312.6 (−0.35%).

V. EXPERIMENTAL RESULTS AND ANALYSIS

As shown in Table I, the submitted predictor achieves a VFS of 0.9921, with IPC of 8.893, CPI of 0.04705, and EPI of 312.6. MPKI is 5.228. Prediction latency is 0.96 cycles, which is below 1 cycle.

A. Replacing Half of the RAM in the *gshareN_ahed* Example Predictor with Two Tagged Tables

After replacing half of its RAM with two tagged tables and adding the training-direction bias and a path-hashed tagged-table index, the submitted predictor reduces MPKI by 22.58% (from 6.930 to 5.365) compared with the *gshareN_ahed* example predictor, at the cost of a 14.85% increase in energy consumption (EPI from 243.1 to 279.2). The GShare RAM size in the submitted predictor is 64 KB; the other 64 KB is more accuracy-efficient when devoted to tagged tables rather than GShare.

Compared with the *tage* example predictor, MPKI is 2.26% lower (5.489 vs. 5.365), prediction throughput is 1.5× higher (IPC 5.769 vs. 8.876), and energy consumption is 4.5× lower (EPI 1265.5 vs. 279.2). The *N*-branch prediction style and large block size inherited from the example *gshareN_ahed* predictor bring the higher prediction throughput and higher energy efficiency, because the tables are read once to generate predictions for 256 instructions rather than for 16 instructions as in the *tage* example. Limiting to only 2 tagged tables leads to area inefficiency (RAM size is 4.2× larger, 130 KB vs. 31 KB), because only two history lengths (20 and 80) are utilized instead of a series of different history lengths.

The two tagged tables (T1 and T2) bring almost all of the VFS score benefit. The training-direction bias and hashing the path into the index together yield a 0.67% MPKI improvement (from 5.401 to 5.365), but only a 0.03% VFS improvement (from 0.9909 to 0.9912).

B. 2-bit Hysteresis

Expanding the hysteresis from 1 bit to 2 bits improves VFS by 0.09% (from 0.9912 to 0.9921) from 2.55% MPKI reduction (from 5.365 to 5.228), at the cost of 11.96% increase in energy consumption (EPI from 279.2 to 312.6) and 35.38% increase in RAM size (from 130 KB to 176 KB).

Compared with the CBP2025 *tage* predictor, MPKI is 18.01% higher (5.228 vs. 4.430) with 23.25% more RAM (176 KB vs. 142.8 KB). To improve area efficiency, additional history lengths should be employed across more tagged tables.

VI. CONCLUSION

Half of the RAM in the example *gshareN_ahed* predictor can be replaced with two tagged tables to achieve accuracy comparable to that of the example *tage* predictor. The *N*-branch prediction style with large blocks provides energy efficiency, while ahead-pipelining combined with RAM-floorplan optimization maintains prediction throughput; together, these techniques can yield a higher VFS.

REFERENCES

- [1] P. Michaud, “A scoring metric for the CBP-NG championship.” <https://github.com/AmpereComputing/cbp-ng/blob/main/docs/vfs.pdf>. Accessed: 2026-05-04.
- [2] Ampere Computing, “CBP-NG: The next generation championship branch prediction simulator.” <https://github.com/AmpereComputing/cbp-ng/>. Accessed: 2026-05-04.
- [3] P. Michaud, “A PPM-like, tag-based branch predictor,” *The Journal of Instruction-Level Parallelism*, vol. 7, p. 10, 2005.
- [4] A. Seznec, S. Jourdan, P. Sainrat, and P. Michaud, “Multiple-block ahead branch predictors,” *Proceedings of the 7th conference on Architectural Support for Programming Languages and Operating Systems*, vol. 30, p. 116–127, Sept. 1996.
- [5] A. Seznec and A. Fraboulet, “Effective ahead pipelining of instruction block address generation,” in *Proceedings of the 30th Annual International Symposium on Computer Architecture*, pp. 241–252, 2003.
- [6] L. Cai, A. Deshmukh, and Y. Patt, “Enabling ahead prediction with practical energy constraints,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pp. 559–571, 2025.
- [7] A. Seznec and P. Michaud, “A case for (partially) TAgged GEometric history length branch prediction,” *The Journal of Instruction-Level Parallelism*, vol. 8, p. 23, 2006.
- [8] A. Seznec, “A 256 Kbits L-TAGE branch predictor,” *Journal of Instruction-Level Parallelism (JILP) Special Issue: The Second Championship Branch Prediction Competition (CBP-2)*, vol. 9, no. 6, 2007.
- [9] A. Seznec, “A 64 Kbytes ISL-TAGE branch predictor,” in *Proceedings of the 3rd Championship Branch Prediction*, 2011.
- [10] A. Seznec, “A new case for the TAGE branch predictor,” in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 117–127, 2011.
- [11] A. Seznec, “TAGE-SC-L branch predictors,” in *Proceedings of the 4th Championship Branch Prediction*, 2014.
- [12] A. Seznec, “TAGE-SC-L branch predictors again,” in *Proceedings of the 5th Championship Branch Prediction*, 2016.
- [13] A. Seznec, “TAGE-SC for CBP2025,” in *Proceedings of the 6th Championship Branch Prediction*, 2025.